

1 L2–3 Supervised Learning

1.1 Network Math

Layer: for batch $X \in \mathbb{R}^{B \times n}$, code convention often uses $Z = XW + b$, $W \in \mathbb{R}^{n \times m}$, output $Z \in \mathbb{R}^{B \times m}$; each column/output unit is a neuron. **Activation** adds nonlinearity: $\text{ReLU} = \max(0, z)$, sigmoid $\sigma(z) = 1/(1 + e^{-z})$. **Binary logistic regression:** $p(y = 1|x) = \sigma(w^\top x + b)$, loss

$$\ell(x, y) = -y \log p - (1 - y) \log(1 - p).$$

Multi-class: logits z_k , probability $p_k = \text{softmax}(z)_k = e^{z_k} / \sum_j e^{z_j}$, **NLL** / cross entropy

$$\ell(x, y) = -\log p_y = -z_y + \log \sum_j e^{z_j}.$$

Gradient fact: $\partial \ell / \partial z_k = p_k - 1[k = y]$.

1.2 CNN

Convolution layer: local receptive field + weight sharing. For input $H \times W \times C$, kernel $K_h \times K_w$, padding P , stride S :

$$H' = \left\lfloor \frac{H + 2P - K_h}{S} \right\rfloor + 1, \quad W' = \left\lfloor \frac{W + 2P - K_w}{S} \right\rfloor + 1.$$

Parameters: $K_h K_w C_{\text{in}} C_{\text{out}}$ + C_{out} . **Pooling** reduces spatial size; **fully connected** consumes flattened/global pooled representation. **CNN** assumption: useful patterns repeat across location, so scanning MLP can be implemented as convolution.

1.3 Optimization and Regularization

Newton step: from second-order Taylor expansion,

$$x^{k+1} = x^k - \eta^k [\nabla^2 f(x^k)]^{-1} \nabla f(x^k).$$

Momentum: $v_{t+1} = \beta v_t + \nabla f(x_t)$, $x_{t+1} = x_t - \eta v_{t+1}$. **Nesterov:** evaluate gradient at lookahead $x_t - \eta \beta v_t$. **AdaGrad:** $G_t = G_{t-1} + g_t^2$, update $x_{t+1} = x_t - \eta g_t / (\sqrt{G_t} + \epsilon)$; learning rate may vanish. **RMSProp:** $s_t = \rho s_{t-1} + (1 - \rho) g_t^2$. **Adam:** first and second moments

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2,$$

$$\hat{m}_t = m_t / (1 - \beta_1^t), \quad \hat{v}_t = v_t / (1 - \beta_2^t), \quad x_{t+1} = x_t - \eta \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon).$$

Dropout: randomly mask units to force redundant features. **Early stopping:** stop when validation loss worsens. **BatchNorm:** normalize per feature in mini-batch

$$\hat{z}_i = \gamma \frac{z_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}.$$

LayerNorm: normalize across hidden dimension of one example, common in sequence models.

2 L4 Energy-Based Models

2.1 Hopfield Network

State $y_i \in \{-1, 1\}$, symmetric W , local field $h_i = \sum_{j \neq i} w_{ij} y_j + b_i$. Update flips neuron if $y_i h_i < 0$. Energy

$$E(y) = -\sum_{i < j} w_{ij} y_i y_j - \sum_i b_i y_i = -\frac{1}{2} y^\top W y - b^\top y$$

decreases under asynchronous updates, because a flip changes energy by $\Delta E = 2y_i h_i < 0$. Finite states imply convergence to a local minimum. Hebbian storage for patterns y^p : $w_{ij} = \frac{1}{p} \sum_p y_i^p y_j^p$, $w_{ii} = 0$.

2.2 Boltzmann Machine

Energy to probability: $P(y) = e^{-E(y)} / Z$, $Z = \sum_{y'} e^{-E(y')}$. Conditional update for binary units:

$$P(y_i = 1|y_{-i}) = \sigma(2h_i)$$

up to the $\{-1, 1\}$ sign convention. **Temperature:** $P_T(y) \propto e^{-E(y)/T}$; annealing lowers T to move from exploration to exploitation. **Maximum likelihood:** for data distribution p_{data} ,

$$\nabla w_{ij} \log P(y) = y_i y_j - \mathbb{E}_{p_\theta} [y_i y_j].$$

Learning = positive phase on data minus negative phase on model samples. Hidden units require $\mathbb{E}_{P(h|v)} [y_i y_j]$ in the positive phase.

2.3 Restricted Boltzmann Machine

RBM: bipartite visible v and hidden h , no intra-layer edges:

$$E(v, h) = -v^\top W h - b^\top v - c^\top h.$$

Conditionals factorize:

$$P(h_j = 1|v) = \sigma(c_j + W_{\cdot j}^\top v), \quad P(v_i = 1|h) = \sigma(b_i + W_{i \cdot} h).$$

Contrastive divergence: start Markov chain at data $v^{(0)}$, sample $h^{(0)}, v^{(1)}, h^{(1)}$, update $W \propto \frac{v^{(0)} h^{(0)\top} - v^{(1)} h^{(1)\top}}{}$.

3 L5 Generative Adversarial Networks

3.1 GAN Objective

Generator $G(z)$ maps noise to samples, discriminator $D(x)$ estimates realness:

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}} \log D(x) + \mathbb{E}_{z \sim p_z} \log(1 - D(G(z))).$$

For fixed G , optimal discriminator

$$D^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)}.$$

Substitution gives $V(D^*, G) = -\log 4 + 2\text{JSD}(p_{\text{data}} \| p_g)$, so the ideal optimum is $p_g = p_{\text{data}}$. Non-saturating generator loss often uses $-\mathbb{E}_z \log D(G(z))$ for stronger gradients.

3.2 WGAN and Stabilization

JSD issue: if supports barely overlap, discriminator becomes perfect and generator gradients vanish.

Wasserstein-1:

$$W(p, q) = \sup_{\|f\|_L \leq 1} \mathbb{E}_{x \sim p} f(x) - \mathbb{E}_{x \sim q} f(x).$$

WGAN critic maximizes $\mathbb{E}_{x \sim p_{\text{data}}} D(x) - \mathbb{E}_z D(G(z))$ under 1-Lipschitz constraint. **Gradient penalty:**

$$\lambda \mathbb{E}_{\hat{x}} (\|\nabla_{\hat{x}} D(\hat{x})\|_2 - 1)^2.$$

Training tricks: alternate critic/generator updates, avoid overpowered discriminator, use normalization, spectral norm, or gradient penalty. Conditional GAN adds label/text/image condition c : $G(z, c)$, $D(x, c)$.

4 L6 Normalizing Flow and VAE

4.1 Normalizing Flow

Invertible map $x = f_\theta(z)$ with tractable Jacobian:

$$\log p_X(x) = \log p_Z(f_\theta^{-1}(x)) + \log \left| \det \frac{\partial f_\theta}{\partial x} \right|.$$

Composition adds log-determinants. Coupling layer splits $x = (x_a, x_b)$:

$$y_a = x_a, \quad y_b = x_b \odot e^{s(x_a)} + t(x_a),$$

inverse is easy and $\log |\det J| = \sum s(x_a)$. 1×1 invertible convolution mixes channels; LU parameterization makes determinant cheap.

4.2 Expectation-Maximization

For latent z , maximize $\log p_\theta(x) = \log \sum_z p_\theta(x, z)$. EM alternates

$$\gamma_i(z) = p_{\theta^{\text{old}}}(z|x_i), \quad \theta^{new} = \arg \max_\theta \sum_i \sum_z \gamma_i(z) \log p_\theta(x_i, z).$$

Jensen view:

$$\log p(x) \geq \mathbb{E}_{q(z)} \log p_\theta(x, z) + H(q),$$

tight when $q(z) = p_\theta(z|x)$.

4.3 Variational Autoencoder

VAE uses prior $p(z)$, decoder $p_\theta(x|z)$, encoder $q_\phi(z|x)$. ELBO:

$$\log p_\theta(x) \geq \mathbb{E}_{q_\phi(z|x)} [\log p_\theta(x|z)] - \text{KL}(q_\phi(z|x) \| p(z)).$$

Reparameterization for Gaussian encoder:

$$z = \mu_\phi(x) + \sigma_\phi(x) \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$$

For diagonal $q = \mathcal{N}(\mu, \sigma^2 I)$ and $p = \mathcal{N}(0, I)$:

$$\text{KL}(q \| p) = \frac{1}{2} \sum_i (\mu_i^2 + \sigma_i^2 - \log \sigma_i^2 - 1).$$

Tradeoff: good likelihood and continuous latent space, but samples may be blurrier than GANs.

5 L7 Diffusion and Flow Matching

5.1 DDPM

Forward noising:

$$q(x_t|x_{t-1}) = \mathcal{N}(\sqrt{\alpha_t} x_{t-1}, \beta_t I), \quad \bar{\alpha}_t = \prod_{s=1}^t \alpha_s,$$

$$q(x_t|x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t) I), \quad x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon.$$

Reverse model $p_\theta(x_{t-1}|x_t) = \mathcal{N}(\mu_\theta(x_t, t), \Sigma_t)$. Predicting noise gives simplified loss

$$L_{\text{simple}} = \mathbb{E}_{t, x_0, \epsilon} \|\epsilon - \epsilon_\theta(x_t, t)\|^2.$$

Sampling:

$$x_{t-1} = \frac{1}{\sqrt{\bar{\alpha}_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(x_t, t) \right) + \sigma_t z.$$

5.2 Score-Based Models and ODE

Score: $\nabla_x \log p_t(x)$. Denoising score matching learns score from noisy data; reverse SDE uses the learned score to transform noise to data. Probability flow ODE has same marginals:

$$dx = \left[f(x, t) - \frac{1}{2} g(t)^2 \nabla_x \log p_t(x) \right] dt.$$

Exact likelihood along ODE:

$$\log p_0(x_0) = \log p_T(x_T) + \int_0^T \text{tr} \left(\frac{\partial f_\theta}{\partial x_t} \right) dt.$$

5.3 Flow Matching

Continuous normalizing flow solves $dx_t/dt = v_\theta(x_t, t)$. Flow matching regresses a vector field:

$$\mathcal{L}_{\text{FM}} = \mathbb{E}_{t, x_t} \|v_\theta(x_t, t) - u_t(x_t)\|^2.$$

Conditional path between noise x_0 and data x_1 often uses $x_t = (1 - t)x_0 + tx_1$, target velocity $u_t = x_1 - x_0$. Conditional flow matching avoids solving likelihood or simulating diffusion during training, and modern text-to-image/video models can use diffusion transformer backbones with flow-matching objectives.

6 L8–9 Sequence Modeling

6.1 RNN and LSTM

Sequence model factorization:

$$p(x_{1:T}) = \prod_{t=1}^T p(x_t|x_{<t}).$$

RNN hidden state $h_t = f_\theta(h_{t-1}, x_t)$; gradients through long chains can vanish/explode. LSTM gates:

$$f_t = \sigma(W_f[h_{t-1}, x_t]), \quad i_t = \sigma(W_i[\cdot]), \quad o_t = \sigma(W_o[\cdot]),$$

$$\tilde{c}_t = \tanh(W_c[\cdot]), \quad c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \quad h_t = o_t \odot \tanh(c_t).$$

Gates create additive memory path, making long-range dependency easier.

6.2 Attention and Transformer

Scaled dot-product attention:

$$\text{Attn}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} \right) V.$$

Multi-head attention runs several projections in parallel. Transformer block = self-attention + MLP + residual + normalization. Positional encoding injects order; causal mask prevents attending to future tokens. Complexity $O(T^2)$ in context length. **Seq2seq:** encoder summarizes source; decoder generates autoregressively. Attention replaces one-vector bottleneck by dynamic alignment over source positions.

7 L10 Modern Large Models

7.1 Pretraining, Scaling, Instruction Tuning

Autoregressive LM: train next-token prediction

$$\max_{\theta} \sum_t \log p_{\theta}(x_t | x_{<t}).$$

Pretraining builds broad capabilities; post-training shapes behavior. Scaling laws empirically fit power laws in model size N , data D , compute C ; compute-optimal training balances parameters and tokens rather than scaling only one axis. **Prompting**: zero-shot/few-shot uses context without parameter updates. Supervised fine-tuning (SFT) trains on instruction-response demonstrations; limitation: one reference answer cannot encode preference among many acceptable responses.

7.2 RLHF and Inference-Time Enhancement

RLHF pipeline: collect demonstrations $\rightarrow$ train SFT policy $\rightarrow$ collect pairwise rankings $\rightarrow$ train reward model $r_{\phi}(x, y) \rightarrow$ optimize policy with KL penalty:

$$\max_{\pi} \mathbb{E}_{y \sim \pi(\cdot|x)} [r_{\phi}(x, y)] - \beta \text{KL}(\pi(\cdot|x) \parallel \pi_{\text{SFT}}(\cdot|x)).$$

Preference optimization can reward uncertainty awareness and penalize hallucination. **RAG**: retrieve external documents, place evidence in context, then generate. Inference-time scaling: spend more test-time compute on search, reasoning, verification, or longer thinking trajectories.

7.3 Multimodal Models

Vision-language models connect visual encoder and LM through projection/cross-attention. Flamingo-style models keep frozen vision/LM modules and insert cross-attention for interleaved image-text few-shot learning. LLaVA-style visual instruction tuning aligns image features with a chat LM. Unified multimodal models aim to handle understanding, generation, editing, and reasoning in one architecture, often with different tokenizers/experts for image and text.